

Altitude offset and ground safety

Round 4 (branch `main`): using the GP in the reference, and giving the certificate an unsafe set

2026-09-27

Table of contents

1	Summary	1
2	The altitude offset	2
3	The ground	2
3.1	What was missing	2
3.2	The unsafe set in the certificate	2
3.3	The backup	3
4	Tuning the reference	3
5	Results	3
6	Update: the certificate is spatial (0.25 m)	4
6.1	The threshold: 0.25 m (default)	5
6.2	Position uncertainty	6
7	Remaining caveats	7

1 Summary

- **Altitude offset (3 m) → fixed.** The reference thrust law ignored the disturbance the GP had learned. Feeding the GP mean forward (`--gp-feedforward`) removes the offset.
- **Ground → part of the certificate.** A plan is certified only while its *whole tube* stays at least `--min-altitude` (0.3 m) above the ground. When no certified plan exists for longer than `--backup-grace` (20 ms), the node hands the vehicle to PX4 **LAND** (`--backup land`).
- **The sim-only $z + 0.5$ adjustment is gone;** the controller uses PX4's altitude.
- With an honest floor, the reference LQR's vertical under-damping became visible. It overshoot the old 0.6 m goal to about 0 m. At your choice, the goal is now **1.0 m** and `Q_ref = diag(40, 10, 40, 40, 3)` (was `diag(40, 5, 40, 5, 3)`).

2 The altitude offset

In steady state (three flights, $t > 30$ s): altitude 3.57 m against a 0.6 m goal, thrust 18.6 N = $mg - 0.98$ N, and learned $\hat{w}_z = -0.98$ N. The *vehicle* was in equilibrium; the *reference* had stopped short. The rollout’s reference input was

$$u_1 = mg - K_{\text{ref}}(x - x_{\text{goal}}),$$

with a p_z gain of 0.32 N/m. In the reference dynamics $\dot{v} = g \cos \theta - \frac{w_y}{m} \sin \theta + \frac{w_z}{m} \cos \theta - \frac{u_1}{m}$, the equilibrium needs $u_1 = mg + w_z$ (at $\theta = 0$), so the LQR term must supply w_z itself: $0.32 e = 0.98$ gives $e = 3.0$ m (observed: 2.97 m). No integral action, no way to reach the goal.

The GP already *knows* w_z ; the reference just didn’t use it. With `--gp-feedforward` (default on):

$$u_1 = mg + (\hat{w}_z \cos \theta - \hat{w}_y \sin \theta) - K_{\text{ref}}(x - x_{\text{goal}}),$$

i.e. the mean disturbance’s component along the thrust axis is cancelled, and the LQR term only has to correct *errors*. $\hat{w}$ is the same GP mean the rollout already evaluates at each step, so this costs nothing extra.

Where does a steady ~ 1 N “vertical wind” come from in SITL? The throttle-to-force map in `sim_utilities.py` disagrees with the simulated motors by about 5 %: the vehicle hovers at a commanded 18.6 N for a 19.6 N weight. The GP learns that model error as a disturbance, which is what it’s for.

3 The ground

3.1 What was missing

The certificate asked one question: *does the tube stay within 1 m of the reference?* A reference that descends through the ground, with a narrow tube around it, passes. Nothing encoded that the ground is unsafe.

3.2 The unsafe set in the certificate

`_row_fails` (and `collection_id_jax`) now also fail a row when the tube’s lowest altitude is below the floor. In NED that means the upper bound of p_z above $-h_{\text{min}}$:

```
(xemb_row[n + 1] > z_max)      # z_max = -min_altitude; xemb_row[n + 1] =  
↪ upper bound of p_z
```

The certification point is the first row that fails *either* test. The early-exit rollout stops there, the safety horizon ends there, and pre-emptive replanning works as before. Offline, for a vehicle 0.6 m up and sinking at 1.5 m/s, certification now ends at the first row whose tube dips below 0.30 m

(0.29 m; the row before is 0.31 m). Without the floor it certified down to 0.21 m. Early-exit and full rollouts still agree exactly.

3.3 The backup

Runtime assurance needs a fallback for when *no* plan can be certified, for example because the vehicle is already too close to the ground for any tube to stay above the floor. Every control tick checks whether the current plan has expired. If the vehicle has been **continuously** without a certified plan for more than 20 ms, the node commands PX4 NAV_LAND, stops publishing RTA commands, and records an event in the flight log (**events/** in the HDF5 file, with the reason).

“Continuously” matters. Near the floor, fresh plans certified for 0 s arrive every ~ 10 ms, so no *single* plan is ever more than ~ 10 ms past its expiry. The first version of the trigger compared against the current plan’s expiry and never fired while the vehicle sat below the floor for 4 s.

4 Tuning the reference

With the floor in place, all five flights with the old tuning were landed by the backup at about 24 s. The logs show why: the *reference* passed the 0.6 m goal still descending at 0.37 m/s and bottomed out near the floor, and the vehicle followed it faithfully. Offline, from the logged state at 2.8 m sinking at 1.5 m/s:

goal	Q_ref (py, pz, h, v,)	lowest reference	settles at
0.6 m	40, 5, 40, 5, 3 (old)	−0.02 m	–
1.0 m	40, 5, 40, 5, 3	0.25 m	–
0.6 m	40, 5, 40, 40, 3	0.31 m	–
0.6 m	40, 5, 40, 80, 3	0.53 m	0.57 m
1.0 m	40, 10, 40, 40, 3	0.66 m	0.98 m

The last row (your choice) leaves 0.36 m between the reference’s lowest point and the floor for the tube.

5 Results

Five SITL flights per tuning (events executor + process backend, all round-3 and round-4 defaults), plus one negative test:

	goal 0.6 m, old Q_ref	goal 1.0 m, new Q_ref	negative test (floor 1.2 m)
completed the RTA phase	0 / 5 (LAND at ~ 24 s)	5 / 5	– (LAND at 23.2 s, as intended)
final altitude (goal)	–	1.05–1.11 m (1.00)	–
lowest altitude	0.30 m (at the floor)	0.98–1.03 m	1.20 m (at the floor)

	goal 0.6 m, old Q_{ref}	goal 1.0 m, new Q_{ref}	negative test (floor 1.2 m)
lowest certified tube	0.30 m	0.85–0.89 m	1.20 m
ticks without a certified plan	0.2–0.3 % (the 20 ms grace)	0.0 %	0.2 %
control rate / max attitude	100 Hz / 30–33°	100 Hz / 31–36°	100 Hz / 31°

In the negative test, certification failed exactly at the 1.2 m floor. The node commanded LAND 20 ms later, and PX4 reported *Landing detected* and *Disarmed by landing*. The backup works end to end; it just isn't needed with the new tuning.

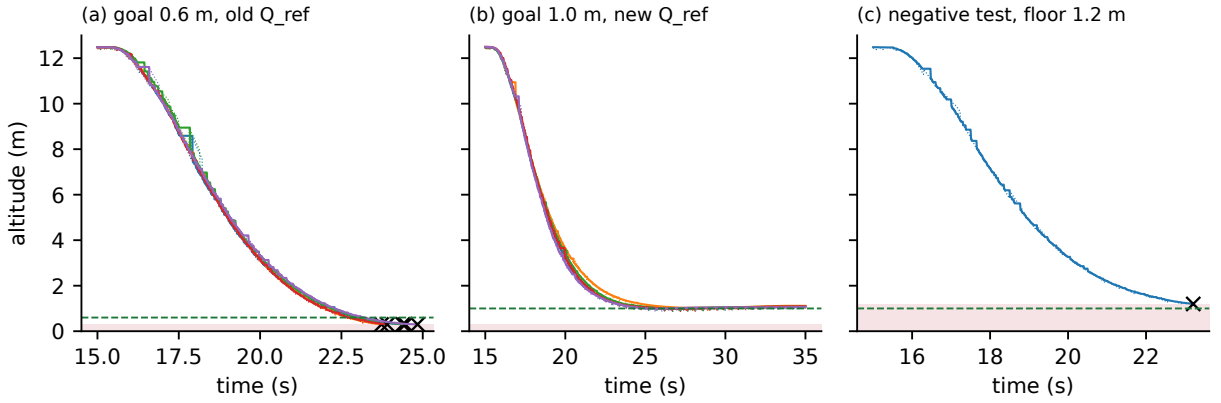


Figure 1: Altitude during the RTA phase (solid) and the lowest altitude of the tube row in use (dotted). Green dashed line: goal; red band: below the floor; $\times$: backup (LAND) engaged. (a) Old tuning: the reference overshoots the 0.6 m goal into the 0.3 m floor, and the backup lands every flight. (b) New tuning: the reference settles at the 1.0 m goal without overshoot, and the certified tube stays about 0.55 m above the floor. (c) With the floor deliberately set above the goal, certification fails at 1.2 m and the backup lands the vehicle.

Per-tick data and backup events: `docs/data/round4/`.

6 Update: the certificate is spatial (0.25 m)

The test applied the threshold to **all five** states of the planar model, each in its own units: 1 m for p_y, p_z , but also 1 m/s for the body-frame velocities h, v and 1 rad for θ . The wind enters as a force, so the velocity bounds widen first; position is their integral and widens later. In the SITL flight all 58 plans ended because the h bound was 1 m/s from the reference, while the position boxes were still a few centimetres to 20 cm wide. The certificate was about velocity, not about where the vehicle can be.

The certificate is now **spatial**. A row fails when a position bound (p_y or p_z , lower or upper) is more than `collection_threshold = 0.5 m` from the reference position. It also fails when the tube reaches the floor, or when any state is NaN (the rollout broke down). With hyper-rectangular tubes,

0.5 m on each axis keeps every certified box inside a 1 m square centred on the reference. Velocity and attitude bounds are not limited.

```
SPATIAL = 2    # py, pz
(jnp.any(jnp.abs(xref_row[:SPATIAL] - xemb_row[:SPATIAL]) > threshold)
 | jnp.any(jnp.abs(xref_row[:SPATIAL] - xemb_row[n:n + SPATIAL]) >
    ↪ threshold)
 | jnp.any(jnp.isnan(xref_row)) | jnp.any(jnp.isnan(xemb_row))
 | (xemb_row[n + 1] > z_max))
```

	all states, 1.0	spatial, 0.5 m (first version)
SITL: what ended the certificates	h (velocity): 58 of 58	y : 38, altitude: 7 (of 45)
SITL: certified horizon, median / min	0.42 / 0.17 s	0.53 / 0.27 s
SITL: plans in 20 s	58	45
SITL: uncertified time	0 %	0 %
SITL: lowest altitude flown / lowest certified tube	0.93 / 0.81 m	1.03 / 0.63 m
SITL: max attitude	32°	27°
numerical, 4 wind scenarios: certified horizon, median	0.49–0.50 s	0.60–0.61 s
numerical: lowest certified tube (strongest wind)	0.52 m	0.36 m (floor 0.3 m)

The horizon grows only by about 25 %: once the velocity bounds are wide, the position bounds follow within about 0.1 s (interval bounds grow roughly exponentially). The tubes now use the space they are allowed: the lowest certified tube is closer to the floor, and the floor test still holds it off. `test/test_sim_analysis.py::test_certificate_is_spatial` checks that a huge velocity/attitude spread alone does not fail a row, and that a 0.6 m position bound or the floor does.

6.1 The threshold: 0.25 m (default)

`--tube-threshold` sets the half-width; the default is now **0.25 m**, so every certified box lies inside a 0.5 m square centred on the reference position. Halving it costs little, because the tube's width grows roughly exponentially along the plan: each halving of the allowed width moves the end of certification only about one doubling time earlier.

Numerical closed loop, 10 runs per threshold (calm, and paper winds $\times 0.3$ / $\times 0.6$ / $\times 1.0$ with 3 seeds):

threshold	completed	certified horizon, median / min	replans	lowest certified tube	plans ended by the floor
0.15 m	10/10	0.48 / 0.33 s	44.5	0.55 m	0
0.25 m	10/10	0.52 / 0.37 s	39.7	0.45 m	0
0.35 m	10/10	0.57 / 0.40 s	36.9	0.35 m	0
0.5 m	10/10	0.60 / 0.42 s	35.1	0.31 m	6

SITL (Python loop 3 flights each, C++ loop 1 flight each): no uncertified time and no backups at either threshold; 0.25 m certifies 0.51 s ahead (median; minimum 0.28–0.39 s) against 0.54 s at 0.5 m, with 44–45 instead of 39 plans in the 20 s RTA phase. A new plan arrives about 10 ms after it is requested, so the shortest horizon still leaves about $28\times$ margin. Tracking is unchanged, and the lowest certified tube stays higher (0.69–0.83 m against 0.54–0.63 m).

6.2 Position uncertainty

The tube is computed from the *estimated* state; the vehicle can be up to the estimation error away from it. Each rollout therefore takes a position uncertainty $\delta = (\delta_y, \delta_z)$ and

- widens the initial box in p_y, p_z to at least δ (so the tube contains the true position), and
- certifies against $0.25\text{ m} + \delta$ per axis (so the tube keeps 0.25 m of room to grow).

`--position-uncertainty` chooses δ : `auto` (default: 0 in simulation, where the estimate is the state the simulator flies, and `ekf2` on hardware), `ekf2`, or a fixed value in metres. With `ekf2`, the node subscribes to `/fmu/out/vehicle_odometry` and uses $\delta = n\sigma$ per axis from EKF2’s `position_variance` ($n = \text{--uncertainty-sigmas}$, default 3, as for the GP bounds). The threshold is a traced input of the compiled rollout, so δ can change every plan without recompiling; each plan stores the δ it used.

EKF2’s variance comes from its configured noise parameters (for motion capture, `EKF2_EVP_NOISE`), not from a measurement of the error: set them to the motion-capture system’s real accuracy, or δ will be too large (or too small).

Stress test. With `--position-uncertainty ekf2` in SITL, EKF2 fuses simulated GPS: $\delta \approx 0.32$ m in y and 0.50 m in z . The flight had no uncertified time, but:

	$\delta = 0$ (default in sim)	δ from EKF2 (GPS-level)
certified horizon, median / min	0.51 / 0.28 s	0.10 / 0.07 s
plans in the 20 s RTA phase	44	244
plans ended by the floor	0	132
lowest certified tube	0.81 m	0.32 m

A wide initial box costs more than the δ it adds to the threshold: mixed-monotone bounds grow faster from a wider box. In altitude, δ also competes with the floor (hovering at 1 m with a ± 0.5 m box leaves 0.2 m above the 0.3 m floor). With motion-capture accuracy (millimetres) neither effect is noticeable; with a badly tuned EKF2 they show up as many more replans and a short horizon.

7 Remaining caveats

- The floor is relative to PX4's local origin (the take-off point). Over uneven ground, or with a different take-off height, `--min-altitude` has to be interpreted accordingly; a range sensor would make it absolute.
- The backup is PX4's LAND. It assumes the ground below is clear; it is not itself certified.
- Only the vertical direction has an unsafe set. Lateral obstacles would enter the same way, as extra conditions in `_row_fails`.